# My overall take

This was a very productive journey. The discovery that the spools were **silently compressing missing time** is the first-order result, and it strongly validates the architecture I described earlier.

It also means that most of v2–v9 were not really model comparisons. They were attempts to solve echo while feeding the algorithms an invalid timeline. WebRTC's AEC architecture assumes a continuous near-end capture stream, a continuous far-end render stream, and a reasonably stable or explicitly reported render-to-capture delay. Arbitrarily deleting time from either stream violates that contract. (WebRTC Git Repositories)

But I think the final "two answers that ship" overcorrect in two important ways:

1. **Mic-only is an excellent speaker-mode fallback, but it should not be the quality ceiling for new aligned recordings.**
2. **Using DTLN-AEC in headphone mode is essentially backwards.** Headphones are the case where you ordinarily do not need AEC at all.

The most important conclusion is:

> **You have now fixed the condition that prevented the architecture I recommended from working, but you have not yet cleanly retested that architecture on the corrected 48 kHz timelines.**

That architecture is still:

```
final = untouched sys + one-way echo-cancelled mic
```

not cross-cancellation, transcript gating, global ducking, or processing both sides.

---

# What your work has conclusively established

## 1. The synchronization bug was real and fundamental

A delay that wanders from 40 ms to 1,433 ms in steps is catastrophic for reference-based cancellation. It explains:

- Why offline FDAF sometimes converged and then failed.
- Why DTLN produced no suppression on old files.
- Why the same DTLN suddenly produced 24 dB suppression on the corrected timeline.
- Why live AEC3 worked substantially better.
- Why progressively more elaborate gating logic never fully solved the problem.

I would slightly revise "live streams are aligned by construction." They are not necessarily sample-aligned or zero-delay. Rather, live processing preserved chronology, and AEC3 continuously received both streams without invisible sections of time being deleted. AEC3 is explicitly designed around render and capture streams with a measurable system delay. (WebRTC Git Repositories)

## 2. The clean sys stream should normally remain pristine

Your v2–v7 tests repeatedly rediscovered this. The remote voice is already available as a clean digital signal. Applying:

- DeepFilterNet,
- cross-cancellation,
- a compressor,

- residual gates,
- or broad ducking

to sys creates damage without solving the local speaker-leakage problem.

The local acoustic problem exists in `mic`, so the normal cancellation direction is:

```
reference: sys
target:    mic
output:    mic_clean
```

Then:

```
final = sys + mic_clean
```

You should only process sys when you have separately demonstrated a **far-end return-echo problem**, discussed below.

## 3. Transcript labels should never determine whether speech exists in the audio

That lesson is exactly right. Diarization and transcription are semantic estimates; audio continuity is a signal problem. Labels can influence UI or speaker attribution, but a mistaken name should never mute the only copy of someone's voice.

## 4. Raw stems and full-band capture were the correct move

Keeping 48 kHz raw sys and raw mic permanently is the right foundation. It makes every later algorithm reversible and allows you to rerender as the stack improves.

---

# Where I disagree with the compressed moral

## 1. Mic-only avoids doubling; it does not solve echo in the full sense

This statement is directionally useful:

> "The mic alone holds one copy of every voice."

But "therefore it must be best" is too strong.

Mic-only removes the **two-copy problem**:

```
clean remote sys
+
delayed remote leakage in mic
```

However, it replaces the pristine remote signal with:

```
remote codec output
→ Mac speakers
→ speaker frequency response
→ room reflections
→ microphone response
→ microphone noise
```

So mic-only can be highly coherent and subjectively pleasant while still losing:

- Remote high-frequency detail.

- Low-volume remote utterances.
- Consistent remote/local level balance.
- Speech when the output volume is very low.
- Clarity in noisy or reverberant rooms.
- Remote speech during local double-talk.

It is an excellent **fail-safe mix** and probably the right permanent answer for old speaker recordings. It should not be the target for newly recorded, correctly timestamped calls.

Also, mic-only can still contain **room reverberation**. It merely avoids creating an additional delayed duplicate by mixing sys back in. Krisp itself distinguishes acoustic echo cancellation from room echo or dereverberation. (Krisp Help)

## 2. The headphone and speaker processing paths are partly inverted

The natural mode matrix is:

| Mode | Recommended playback |
|---|---|
| Headphones | `sys + raw/lightly-denoised mic`; bypass AEC |
| Speakers, aligned, AEC working | `sys + AEC-cleaned mic` |
| Speakers, AEC uncertain | Mic-only fallback |
| Old desynchronized speakers recordings | Mic-only |
| Hybrid route changes | Per-segment selection with crossfades |

DTLN-AEC expects a microphone signal and a loopback/far-end signal so that it can remove playback leakage from the microphone. Its own test convention uses `mic` and `lpb` inputs. In headphone mode, that physical echo path is ordinarily absent, so an AEC model has nothing useful to cancel; any material change it makes to your voice is likely collateral damage. (GitHub)

For headphone playback, I would initially use:

```
sys untouched
+
mic raw
```

Optionally:

```
sys untouched
+
DeepFilterNet(mic raw, conservative)
```

No DTLN-AEC. No sidechain duck. No transcript gating.

If the mic contains almost no sys coherence, the echo canceller should be bypassed completely.

## 3. Sixteen kilohertz was a limitation, but probably not the sole “tinny” root cause

The 48 kHz archive was absolutely the correct fix. A 16 kHz sample rate removes everything above 8 kHz and cannot reproduce full-band detail.

But 16 kHz alone does not normally produce:

- Robotic modulation.
- Phasey doubling.
- Rapid tonal changes.

- Musical-noise artifacts.
- Pumping.

Those are more characteristic of aliasing, hard gates, residual suppression, cross-cancellation, or dynamic compression.

There is a useful reality check: Krisp currently says its virtual speaker is mono and limited to 16 kHz, yet you find its sound warm and natural. (Krisp Help)

So the experiment:

> DFN-on is tinny, DFN-off is tinny

showed that DFN was not the only problem. It did **not** isolate bandwidth as the sole problem. The unfiltered 48→16 kHz decimation bug could itself create severe in-band aliasing, while the cross-cancellation, residual gate, and 8:1 ducking were also likely major contributors.

The correct conclusion is:

> Full-band recording was necessary, but naturalness also requires avoiding time-varying suppression and modulation artifacts.

---

# The correct experiment has not yet really been run

Your second 48 kHz AEC3 instance was tried and reverted **before** the decisive spool-alignment discovery. The DTLN v8/v9 attempts were also performed on the desynchronized recordings. The post-fix test proved DTLN could achieve 24 dB suppression, but it does not sound like you then rendered the simple, correct speaker-mode mix and compared it cleanly.

I would now run this exact blind comparison on several newly aligned speaker calls:

### A — Coherent fallback

```
mic_raw only
```

### B — Correct DTLN mix

```
sys_raw
+
DTLN_AEC(mic_raw, reference=sys_raw)
```

### C — Correct AEC3 mix

```
sys_raw
+
AEC3_conservative(mic_raw, reference=sys_raw)
```

### D — Hybrid full-band mix

```
sys_raw
+
48 kHz linear AEC
+
conservative residual echo suppressor on mic only
```

For all four:

- Do not process sys.
- Do not duck sys.
- Do not use transcript spans.
- Do not use the old residual gate.
- Match perceived loudness before comparing.
- Include remote-only, local-only, and genuine double-talk segments.

That comparison will answer the real question.

A 24 dB suppression number is promising, but it is not enough by itself. Suppose the original speaker leakage in mic was 12 dB below sys; another 24 dB suppression leaves the duplicate approximately 36 dB below sys, which may be effectively inaudible. But if the original leakage was nearly as loud as sys, a −24 dB residual may still be audible during pauses.

Evaluate the final residual relative to sys, not just DTLN's internal suppression ratio.

Microsoft's AEC Challenge selects systems using subjective mean-opinion quality and word accuracy across single-talk and double-talk scenarios—not echo attenuation alone. (GitHub)

---

# Why AEC3 made your voice robotic

I do not think that result falsifies AEC3. It falsifies using its default, communication-oriented residual suppression as an archival mastering processor.

AEC systems contain two conceptually different stages:

```
1. Linear adaptive cancellation
2. Nonlinear residual suppression
```

The linear stage estimates the speaker-to-microphone path and subtracts its echo estimate. It tends to be relatively transparent when converged.

The residual suppressor then attenuates time-frequency regions believed to contain leftover echo. This is where:

- Metallic consonants,
- under-water sound,
- double-talk damage,
- and robotic modulation

typically appear.

Current AEC3 exposes extensive controls for filter length, delay behavior, near-end tuning, dominant near-end detection, high-frequency suppression, comfort noise, and other suppression behavior. (WebRTC Git Repositories)

I would test two approaches:

### Practical approach

Use AEC3 for:

- Live transcription.
- Delay estimation.
- Echo-state metrics.
- A comparison baseline.

Use a separate 48 kHz partitioned frequency-domain adaptive filter plus a much milder residual postfilter for final

archival playback.

### More difficult approach

Integrate AEC3 below the simple APM wrapper and tune the `EchoCanceller3Config` specifically for archival naturalness:

- Very conservative near-end suppression.
- Strong dominant-near-end protection.
- Less high-frequency attenuation.
- Longer convergence allowance.
- Route-specific resets.
- No AGC or generic WebRTC NS.

Pin the WebRTC commit because these internal interfaces evolve.

WebRTC main has also been adding integration for a neural residual echo estimator, including multichannel handling and APM integration tests during 2025–2026. It is an interesting experimental branch for you, but I would not make an active upstream-development feature your first shipping dependency. (WebRTC Git Repositories)

---

## Use a confidence crossfade rather than a hard per-meeting choice

The ideal architecture does not have to choose “clean two-stem mix” or “mic-only” once for the entire recording.

Construct two complete mixes:

$$
Y_{\text{clean}} = S_{\text{sys}} + M_{\text{AEC}}
$$

$$
Y_{\text{fallback}} = M_{\text{raw}}
$$

Then:

$$
Y = \alpha Y_{\text{clean}} + (1-\alpha)Y_{\text{fallback}}
$$

where $\alpha$ is based on AEC confidence.

- $\alpha = 1$: AEC is converged; use clean sys plus clean mic.
- $\alpha = 0$: alignment or suppression is unreliable; use coherent mic-only.
- Intermediate values: equal-power crossfade over perhaps 100–500 ms.

This is importantly different from mixing `sys + mic_raw`. You are crossfading between **two complete candidate recordings**, so you never leave the raw speaker leakage fully mixed with sys for long.

Useful confidence inputs include:

- Sys-to-mic coherence at the estimated delay.
- AEC residual echo estimate.
- Delay-estimator confidence.
- Whether the route just changed.

- Far-end-only versus near-end-only versus double-talk state.
- Output-volume changes.
- Callback discontinuities.
- Whether near-only speech was materially changed.

For near-end-only periods where sys is silent, the result should be almost exactly the raw mic. That gives you a very strong transparency invariant:

> When no echo source exists, the echo processor should not audibly alter the local speaker.

---

# Separate the two possible echo problems

Your writeup may be combining two distinct effects.

## Local speaker leakage

Remote participant speaks:

```
sys → your speakers → your room → your mic
```

Correct treatment:

```
AEC with sys as reference and mic as target
```

## Returned-self or far-end echo

You speak, your voice travels to the other participant, their speaker leaks into their microphone, and your voice returns in sys.

Correct conceptual treatment:

```
AEC or targeted suppression with your clean local mic as reference and sys as target
```

That second path may contain:

- Network jitter.
- Codec transformations.
- The remote room.
- The remote speaker and microphone.
- The remote conference client's own AEC.

It is much harder than local AEC.

Before keeping any sys duck, run two controlled tests:

1. **Remote-only test:** you are silent while remote audio plays. This measures local sys→mic leakage.
2. **Local-only test:** remote side is silent while you speak. Any copy of you in sys is returned-self echo.

If test 2 shows no meaningful self-echo, eliminate sys ducking entirely.

If test 2 does show self-echo, duck only when the sys signal is demonstrably correlated with a delayed copy of your local mic—not simply whenever local VAD says you are speaking. Otherwise you destroy legitimate remote interjections and double-talk.

An 8:1 sidechain duck is a podcast-style aesthetic effect, not faithful call reconstruction. A mild optional “conversation focus” mode could use it, but it should not be the canonical recording.

---

# One capture detail I would verify immediately

The phrase:

> “stamps every spool write with the buffer’s host-clock time”

could describe either the correct thing or a subtle remaining bug.

## Correct

Use the source buffer’s media timestamp:

```
CMSampleBuffer presentation timestamp
+
sample count / sample duration
```

## Risky

Call `mach_absolute_time()` when the callback runs or immediately before writing.

Callback arrival time is not capture time. If the processing queue stalls for 80 ms but the sample buffers themselves remain contiguous, arrival-time stamping can make you insert 80 ms of false silence.

Core Media exposes each sample buffer’s presentation timestamp and total duration specifically for reconstructing its media timeline. (Apple Developer)

For each stream, do something like:

```
startFrame = round((bufferPTS - sessionEpoch) * 48000)
endFrame   = startFrame + bufferFrameCount
```

Then:

- Insert silence when `startFrame > previousEndFrame`.
- Resolve overlap when `startFrame < previousEndFrame`.
- Treat tiny fractional discrepancies as resampling/drift, not arbitrary silence.
- Record discontinuities as metadata.

## Strong Apple-specific improvement

On macOS 15 and later, ScreenCaptureKit can capture both system audio and microphone audio from the same `SCStream`. Apple’s current sample enables `captureMicrophone` and installs `.audio` and `.microphone` stream outputs on the same capture session.

I would benchmark:

```
configuration.capturesAudio = true
configuration.captureMicrophone = true

addStreamOutput(..., type: .audio, ...)
addStreamOutput(..., type: .microphone, ...)
```

against your current SCK-plus-separate-mic-tap design.

It may provide a cleaner shared media-time foundation. It will not remove the physical acoustic delay—your measured 126 ms remains legitimate—but it can eliminate much of the cross-framework clock reconciliation.

Also, keep only one acquisition source of truth per side:

```
sys 48 kHz master
mic 48 kHz master
```

Derive 16 kHz transcription frames from those masters with a stateful resampler. Avoid maintaining independently clocked 16 kHz and 48 kHz capture spools.

---

# Your mode detector needs one more pass

The summary contains an apparent numerical inconsistency:

- Quiet speakers: `1.03`
- Normal speakers: `0.57`
- Headphones: `0.004`
- Threshold: `0.15`

As written, both speaker cases exceed the threshold, so the quiet-speaker case would not have slipped under it. Those may be post-fix measurements, or the summary may have inverted one quantity. I would resolve that before treating the detector as settled.

More importantly, classify echo path using:

1. **Output route metadata** as a strong prior.
2. **Delayed normalized coherence or correlation** between sys and mic as direct evidence.
3. Energy ratio only as a supporting feature.

A good signal score is approximately:

```
maximum normalized sys↔mic coherence
over plausible acoustic delays
during sys-active, local-speech-light windows
```

Use several seconds of evidence and hysteresis. This distinguishes:

- Quiet speakers from headphones.
- Speaker volume changes.
- External monitors.
- Bluetooth devices.
- Hybrid route changes.

The actual question is not merely “headphones or speakers?” It is:

> “Is there currently a measurable acoustic path from sys into mic?”

That is the fact the processing pipeline cares about.

---

# Revised model ladder for your exact system

## Live transcription

Keep the current AEC3 path. It is already working and transcription benefits more from aggressive echo removal than archival playback does.

```
sys 48 kHz
mic 48 kHz
→ timestamp alignment
→ downsample
→ AEC3 at 16 kHz
→ transcription
```

## Headphone playback

```
sys raw 48 kHz
+
mic raw or conservative DeepFilterNet 48 kHz
```

No DTLN-AEC unless measurable leakage exists. No default duck.

## Speaker playback, first candidate

```
sys raw 48 kHz
+
conservatively configured AEC3 mic
```

Retest now that the timelines are fixed.

## Speaker playback, likely best open architecture

```
sys raw 48 kHz
+
48 kHz linear PBFDAF mic cancellation
+
small residual echo mask
+
optional conservative DeepFilterNet
```

This gives you direct control over the part that was making AEC3 robotic.

## Role for DTLN-AEC

DTLN's post-fix 24 dB result is highly valuable: it proves the reference and mic are now coherent enough for neural AEC.

I would use DTLN as:

- A speaker-mode AEC benchmark.
- A residual-echo activity estimator.
- Possibly a low-band suppression-mask generator.
- A fallback for ASR-clean audio.

I would not make its 16 kHz waveform the canonical full-band playback output, and I would not run it by default in headphone mode.

## Advanced experimental path

Test the newer AEC3 neural-residual-estimator integration or train a small custom residual model using:

```
sys reference
raw mic
linear echo estimate
linear AEC residual
near/far/double-talk indicators
```

This is much closer to a Krisp-like hybrid than running a monolithic DTLN model on everything.

## Virtual speaker

A virtual speaker is no longer the immediate next step. The corrected timestamps may get you most of the way there for post-call rendering.

It remains the eventual highest-control architecture because it gives you the exact render buffers inline, which is also how Krisp positions itself: a virtual layer between the conferencing application and the physical microphone/speaker, with both Krisp devices required for acoustic echo cancellation. (Krisp Help)

I would defer the driver until you have tested the corrected aligned 48 kHz AEC mix across:

- Built-in speakers.
- External monitors.
- USB speakers.
- Bluetooth.
- Device changes.
- Long, CPU-loaded calls.

---

# What I would ship at this moment

For **old speaker recordings**, mic-only is the right product answer.

For **new headphone recordings**, ship:

```
untouched sys + raw/lightly-denoised mic
```

with no DTLN-AEC and no default duck.

For **new speaker recordings**, keep mic-only as the safe fallback, but immediately run the corrected one-way AEC experiment. Once it clears blind listening tests, use the confidence-crossfade architecture:

```
high confidence → sys + clean mic
low confidence  → mic-only
```

For **live transcription**, keep AEC3 exactly where it is.

The journey did not disprove clean two-stem playback. It disproved **desynchronized AEC, bidirectional cross-cancellation, transcript-controlled gating, and aggressive residual suppression**. Now that the timeline is correct, the simple architecture you originally wanted is finally testable on fair terms.